МИНИСТЕРСТВО ЦИФРОВОГО РАЗВИТИЯ, СВЯЗИ И МАССОВЫХКОММУНИКАЦИЙ РОССИЙСКОЙ ФЕДЕРАЦИИ

ФЕДЕРАЛЬНОЕ ГОСУДАРСТВЕННОЕ БЮДЖЕТНОЕ ОБРАЗОВАТЕЛЬНОЕУЧРЕЖДЕНИЕ ВЫСШЕГО ОБРАЗОВАНИЯ «САНКТ-ПЕТЕРБУРГСКИЙ ГОСУДАРСТВЕННЫЙ УНИВЕРСИТЕТ ТЕЛЕКОММУНИКАЦИЙ ИМ. ПРОФ. М.А. БОНЧ-БРУЕВИЧА» (СПбГУТ)

ОТЧЕТ ПО ИТОГОВОЙ РАБОТЕ

«РАЗРАБОТКА ВЕБ-СЕРВИСА БРОНИРОВАНИЯ РЕСУРСОВ»
(тема отчета)

09.02.07 Информационные системы и программирование. Разработчик веб и мультимедийных приложений
(код и наименование направления/специальности)

Студенты: Шумская Э.О, Меладзе Ю.А., группа К841

Проверила: Кривоносова Н.В., преподаватель

Цель работы:

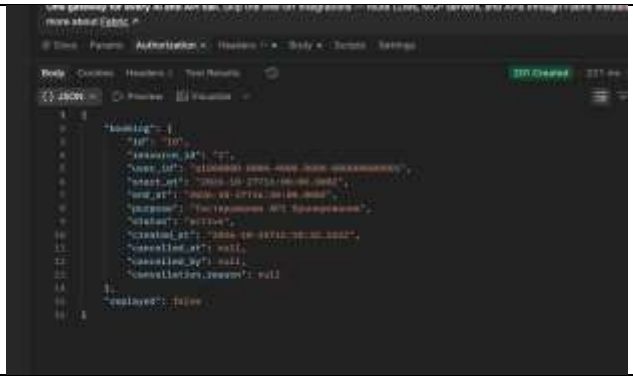
Разработать и развернуть защищённый веб-сервис бронирования ресурсов. Обеспечить корректность операций при одновременных запросах, разграничение доступа, понятный интерфейс и возможность восстановления данных. Объединить ранее изученные технологии (JWT, CI/CD) с проектированием базы данных, разработкой серверных функций и интерфейса.

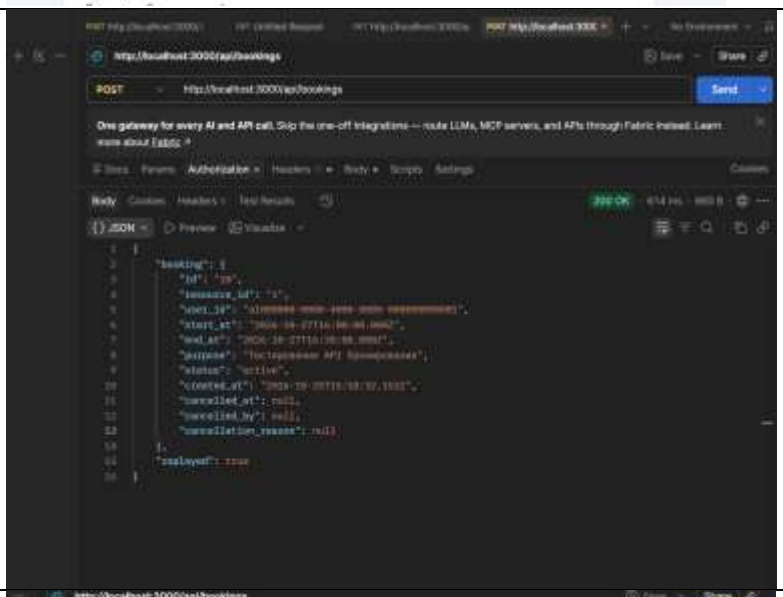
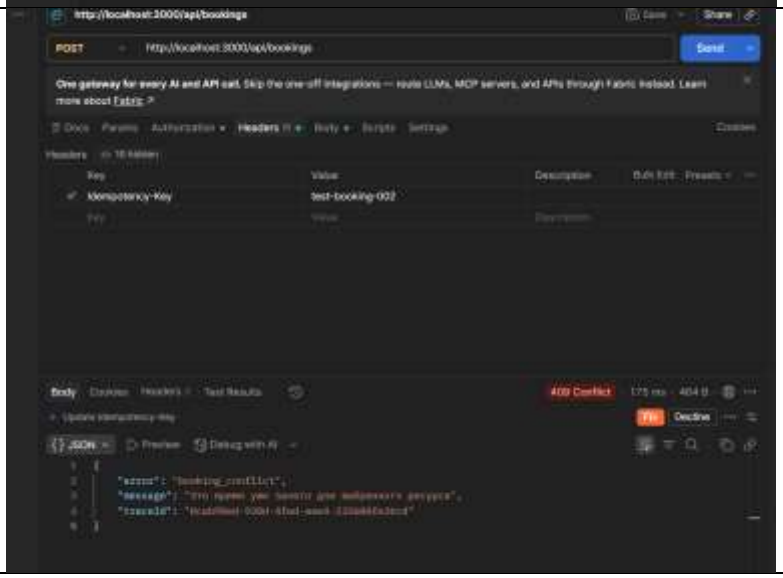
1. **Описание проекта:**

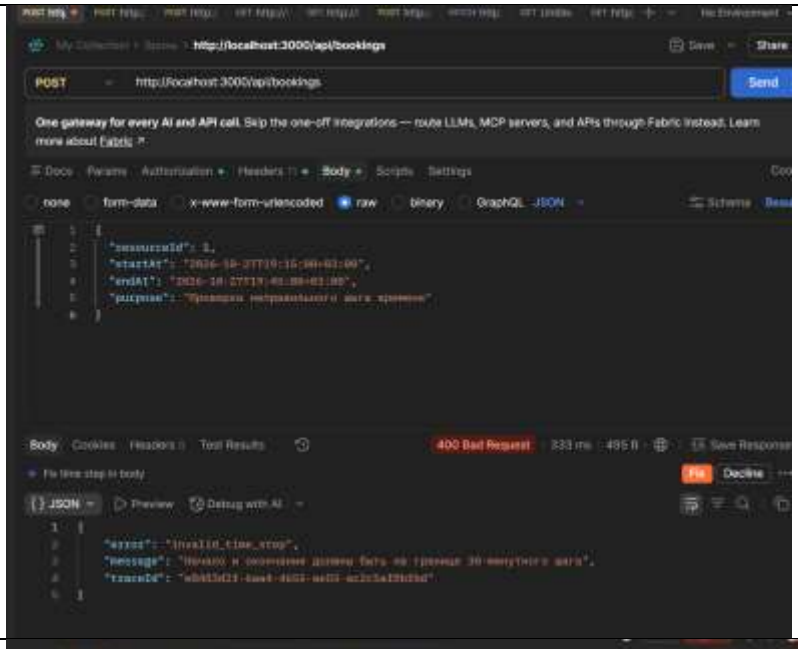
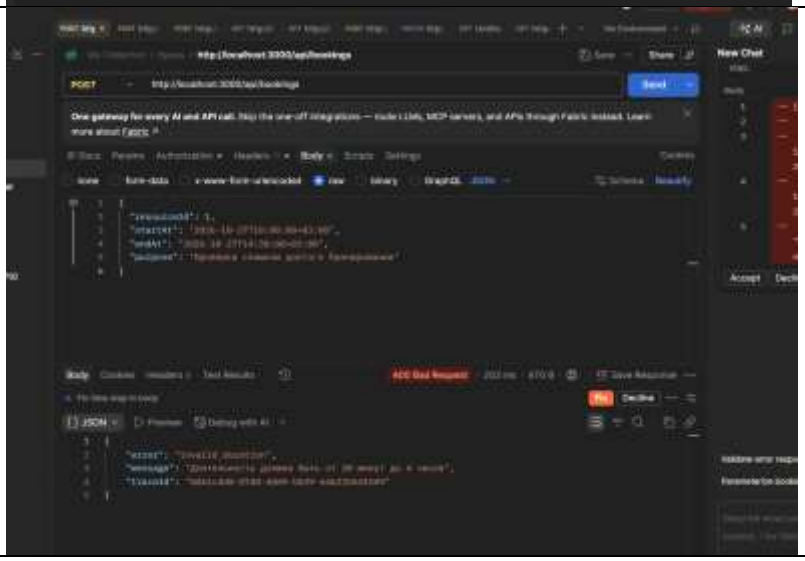
«Бронь без конфликтов» – веб-сервис для бронирования ресурсов образовательной организации. Пользователь может посмотреть доступные ресурсы, выбрать время и оформить бронь. Администратор может управлять ресурсами, просматривать бронирования, отменять их и смотреть журнал аудита.

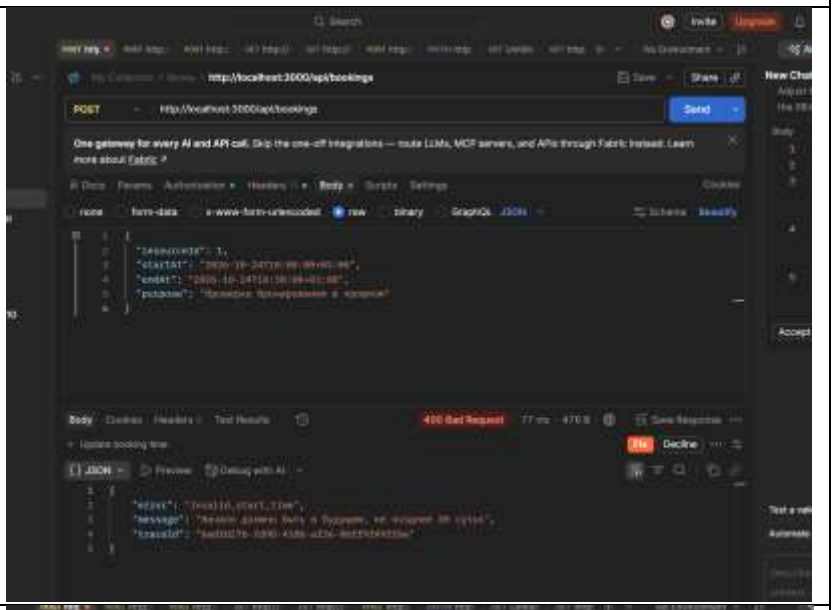
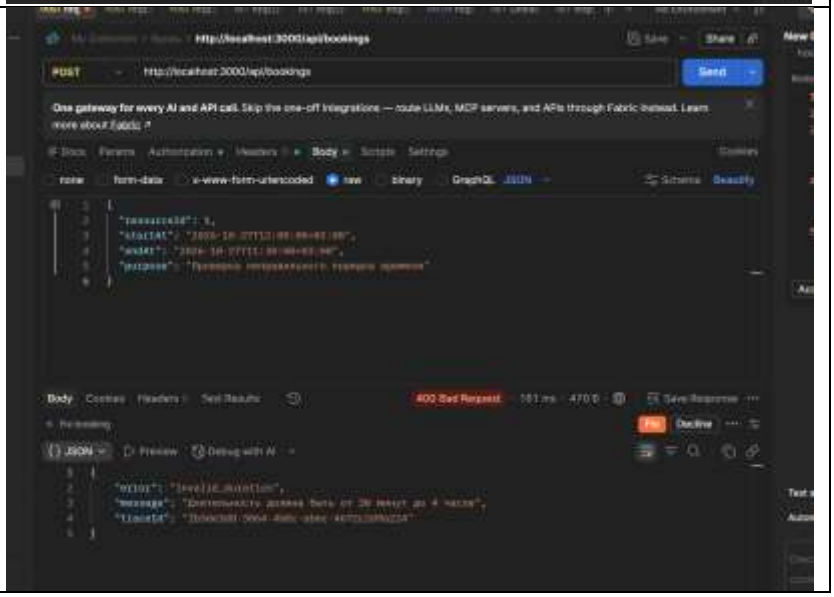
2. **Технологии и инструменты разработки**
   - Backend (серверная часть): Node.js, Express.
   - База данных: PostgreSQL.
   - Работа с БД: DBeaver.
   - Frontend (клиентская часть): HTML, CSS, JavaScript.
   - Проверка API: Postman.
   - Разработка: Visual Studio Code.
3. **Роли пользователей**
   - Пользователь: просмотр каталога ресурсов, создание бронирований для себя, просмотр собственных записей, отмена своей будущей записи.
   - Администратор: управление ресурсами (создание, редактирование, отключение), просмотр всех записей и их владельцев, отмена чужой будущей записи с указанием причины, просмотр журнала аудита действий.
4. **Протокол ручной проверки**

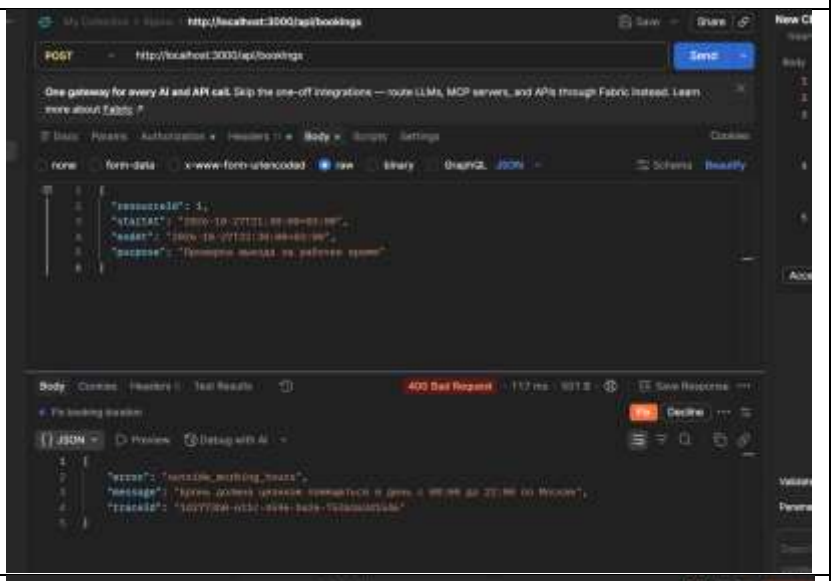
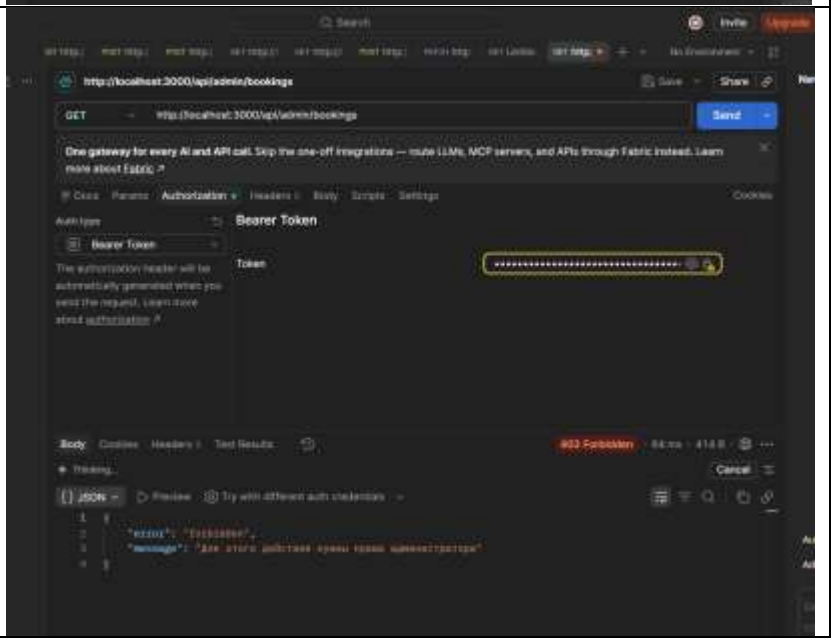
Таблица 1. Ручная проверка

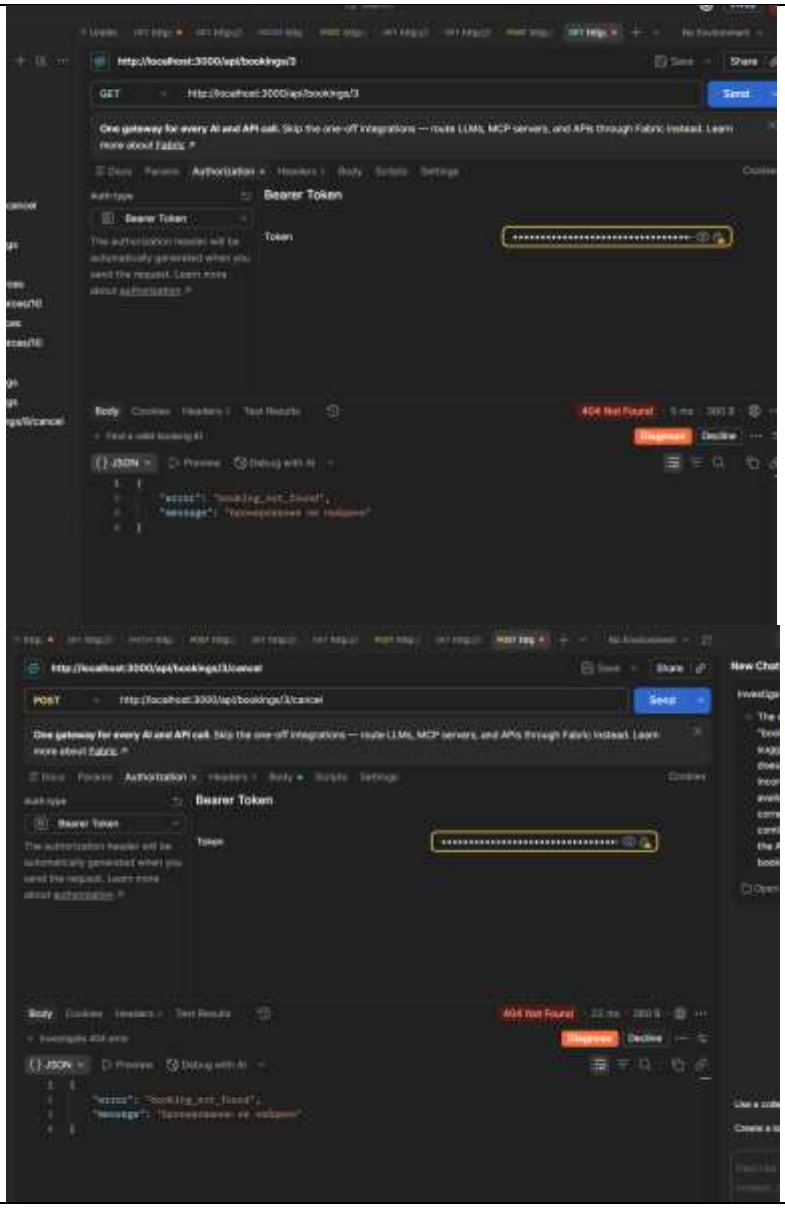
№	Проверка	Ожидаемый результат	Откуда взять скриншот (Фактический результат)
1	Успешное создание бронирования	Код 201	

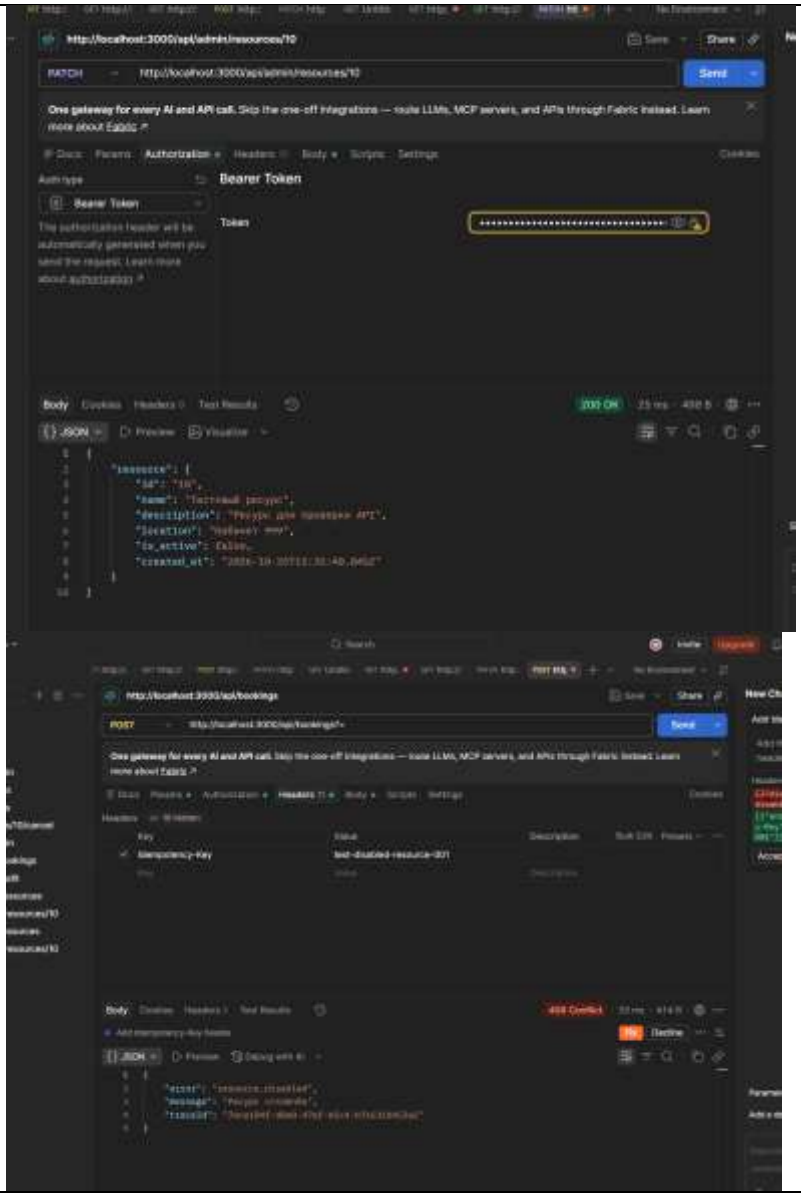
№	Проверка	Ожидаемый результат	Откуда взять скриншот (Фактический результат)
			
2	Повтор запроса с тем же Idempotency-Key	Код 200, та же бронь	
3	Пересечение времени (конфликт)	Код 409 Conflict	

№	Проверка	Ожидаемый результат	Откуда взять скриншот (Фактический результат)
4	Нарушение шага времени (30 мин)	Код 400	
5	Длительность более 4 часов	Код 400	

№	Проверка	Ожидаемый результат	Откуда взять скриншот (Фактический результат)
6	Бронирование в прошлом	Код 400	
7	Окончание раньше начала	Код 400	

№	Проверка	Ожидаемый результат	Откуда взять скриншот (Фактический результат)
8	Бронь вне рабочего времени (ночью)	Код 400	
9	Доступ обычного юзера в админку	Код 403 Forbidden	

№	Проверка	Ожидаемый результат	Откуда взять скриншот (Фактический результат)
10	Просмотр/отмена чужой брони	Код 404 Not Found	

№	Проверка	Ожидаемый результат	Откуда взять скриншот (Фактический результат)
11	Бронирование отключенного ресурса	Ошибка (Код 400/409)	

5. **Журналирование (traceId)**
   - Получение токена (Рис. 1).

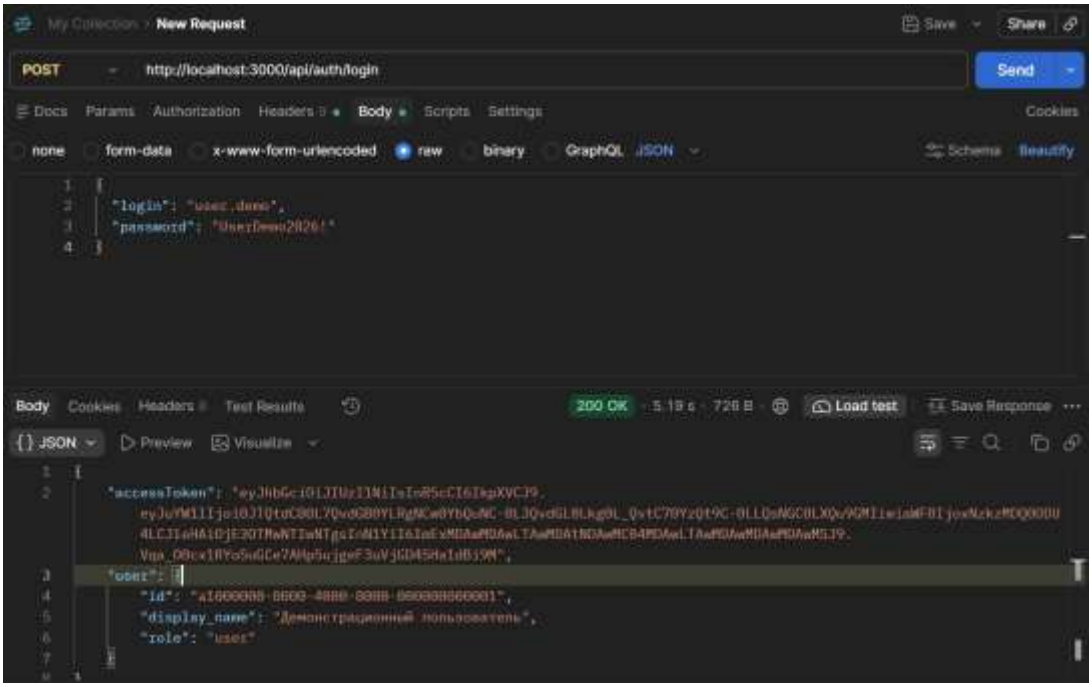


Рисунок 1. Получение токена

- Создание ошибочного запроса (Рис. 2).

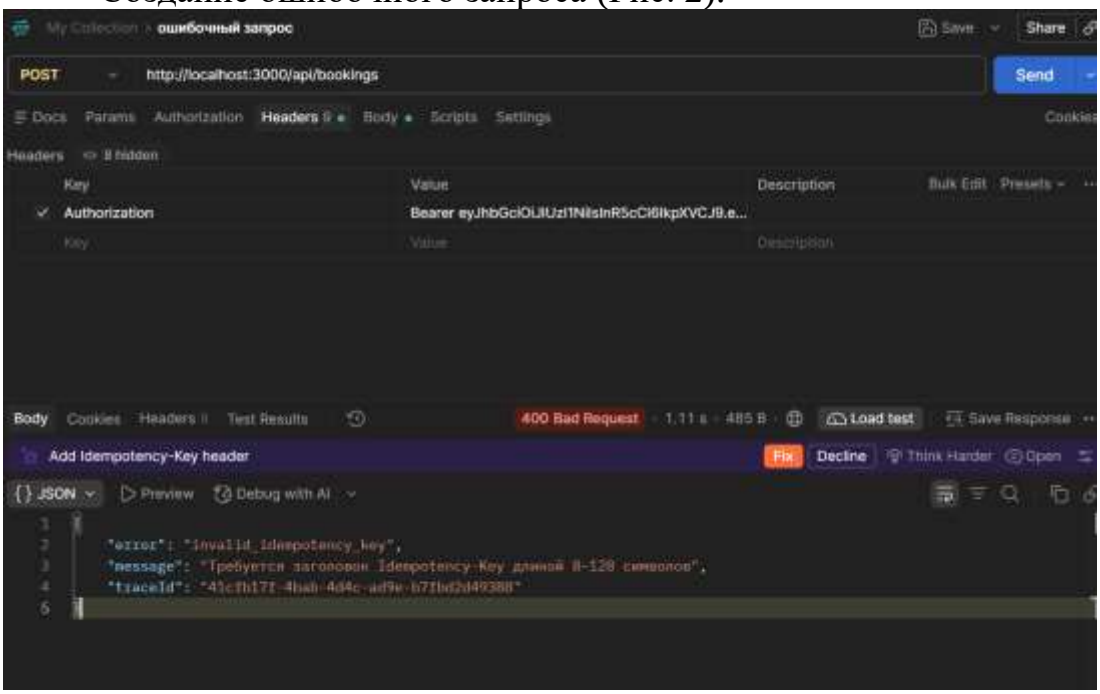


Рисунок 2. Создание ошибочного запроса

- Проверка его в логах. В папке logs, в файле app.log появилась строчка:

```
 (C:\\Users\\yulia\\OneDrive\\Документы\\GitHub\\bron-booking-
service\\node_modules\\router\\index.js:291:5)","timestamp":"2026-10-26
23:05:14","traceId":"41cfb17f-4bab-4d4c-ad9e-b7fbd2d49388"}
```


6. **CI/CD (GitHub Actions)**

   Проверка работы с отображением результатов в действиях на GitHub-е (Рис. 3).

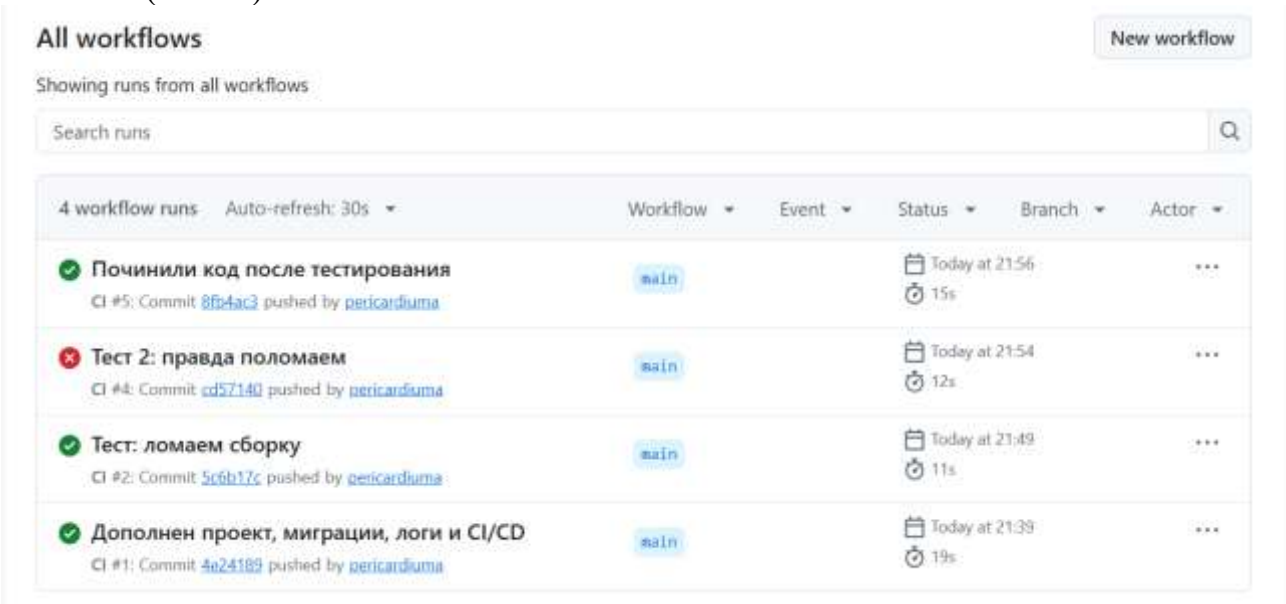


Рисунок 3. Действия над проектом

7. **Описание основных технических решений**

- Защита от конкурентных запросов (гонки данных)

При создании бронирования используется транзакция с блокировкой строки ресурса (SELECT ... FOR UPDATE). Это гарантирует, что если два пользователя одновременно попытаются забронировать один и тот же ресурс на одно и то же время, сервер обработает запросы последовательно. Первый запрос успешно создаст бронь, а второй получит ошибку 409 Conflict.

- Защита от дублирования запросов (Идемпотентность)

Для защиты от двойных нажатий кнопки или потери ответа сети клиент передает заголовок Idempotency-Key. Сервер сохраняет этот ключ в таблице idempotency_keys вместе с результатом. Если запрос с таким же ключом придет повторно, сервер вернет уже созданную бронь, не создавая дубликат.

- Разграничение прав доступа (Авторизация)

Проверка ролей и принадлежности записей выполняется на стороне сервера. Владелец брони определяется по данным из JWT-токена, а не по тому, что пришлет клиент. Обычный пользователь не может получить доступ к чужой записи (возвращается 404, чтобы не раскрывать факт её существования), а административные маршруты защищены проверкой роли (403 Forbidden).

- Логирование и трассировка

Для диагностики ошибок используется библиотека winston. При возникновении ошибки серверу присваивается уникальный traceId, который возвращается пользователю и записывается в лог-файл. Это позволяет быстро найти причину сбоя по номеру ошибки.

Вывод:

В ходе выполнения работы был успешно разработан веб-сервис «Бронь без конфликтов».

В процессе работы были изучены и применены на практике механизмы работы с транзакциями PostgreSQL, что обеспечило надежную защиту от конкурентных запросов (гонки данных) при бронировании – один ресурс невозможно забронировать на пересекающееся время. Реализована защита от дублирования запросов с помощью ключей идемпотентности (Idempotency-Key), а также настроено разграничение прав доступа (JWT-токены, проверка ролей на сервере). Дополнительно был настроен конвейер CI/CD (GitHub Actions), который автоматически проверяет синтаксис кода при каждом пуше и блокирует сборку при наличии ошибок. Проверено резервное копирование и успешное восстановление базы данных в тестовую среду.

Все требуемые сценарии (создание, отмена, обработка конфликтов, разграничение прав) работают корректно. Цель работы достигнута в полном объеме.